Zadanie 1 - Kamera 3D

Grafika Komputerowa

Martyna Kochalska, 331704

1. Cel projektu **1**
2. Wybrane technologie **1**
3. Sterowanie **2**
4. Użyte algorytmy **2**
Wybrany układ 2
Dzielenie perspektywiczne 2
Translacja i obracanie 3
5. Testy **4**

1. Cel projektu

Celem projektu była implementacja wirtualnej kamery 3D, umożliwiającej wykonanie 12 operacji: 6 ruchów translacyjnych (przód, tył, góra, dół, lewo, prawo) oraz 6 obrotów (po 2 wzdłuż osi X, Y i Z). Dodatkowo zaimplementowano 2 funkcje przybliżania i oddalania sceny.

Na pierwszym etapie prac wprowadzono, zgodnie z zaleceniami prowadzącego, następujące uproszczenia:

- Brak eliminacji obiektów zasłoniętych oraz przycinania.
- Brak wypełnienia figur - rysowane są wyłącznie krawędzie.
- Figury nie przenikają się, mogą jedynie stykać się ze sobą.

W ramach sceny wygenerowano zestaw 4 prostopadłościanów imitujących "domki".

2. Wybrane technologie

Do stworzenia aplikacji wykorzystano język Python wraz z biblioteką Pygame, służącą do rysowania punktów i linii, oraz NumPy do wykonywania obliczeń macierzowych. Kod napisano i uruchomiono w środowisku Visual Studio Code.

3. Sterowanie

Translacja:

```
Lewa strzałka / Prawa strzałka - ruch w lewo / prawo
Górna strzałka / Dolna strzałka - ruch do przodu / do tyłu
Shift / Spacja - ruch w dół / w górę
```

Obrót:

```
W / S - pochylenie (oś X)
A / D - odchylenie (oś Y)
Q / E - przechylenie (oś Z)
```

Pozostałe:

```
= / -  - przybliżenie / oddalenie
ESC - wyjście z programu
```

4. Użyte algorytmy

Wybrany układ

W zaimplementowanym rozwiązaniu zastosowano model, w którym wirtualna kamera jest statyczna i zawsze znajduje się w początku układu współrzędnych (punkt 0, 0, 0). Zarówno operacje ruchu jak i obrotu realizowane są poprzez odpowiednie przekształcanie całej sceny w kierunku odwrotnym do pożądanego wektora ruchu kamery.

Dzielenie perspektywiczne

Punkty zapisane w macierzach generowane są dla trójwymiarowej przestrzeni XYZ. Aby wyświetlić to jako obraz na płaskim monitorze, należy je przepisać na przestrzeń dwuwymiarową. Aby to zrobić należy punkty X i Y podzielić przez pozostały punkt Z. Dodatkowo, jest to dobry moment na uwzględnienie przybliżenia poprzez przemnożenie go przez koordynaty wierzchołka.

```
x_proj = (x * zoom) / z
y_proj = (y * zoom) / z
```

W bibliotece Pygame punkt 0,0 na ekranie zaczyna się w lewym górnym rogu okna, zamiast samego środka, dla którego obliczenia to były wykonane. Należy więc przesunąć każdy wyliczony wyżej wierzchołek i obliczyć jego nową lokalizację poprzez przesunięcie go o połowę szerokości/wysokości okna oraz inwersję osi Y.

```
x_ekran = x_proj + (WIDTH / 2)
y_ekran = -y_proj + (HEIGHT / 2)
```

okna oraz inwersję osi Y.

Przy wykorzystanej metodzie, aby program działał prawidłowo, potrzebne były 2 zabezpieczenia. Najważniejsze z nich - wartość „Z” nigdy nie może wynosić 0, inaczej program przestanie działać, gdy napotka dzielenie.

Po drugie, podczas poruszania się do przodu zmniejszana jest wartość Z, co w pewnym momencie może spowodować wejście w wartości ujemne. Będzie to powodowało, że wierzchołki będą nieprawidłowo generowane nie tylko w założonych pozycjach, ale również na ich ujemnych odpowiednikach, co powoduje błędy graficzne. Przypadek ten został przedstawiony w rozdziale 5. „Testy”.

Aby to naprawić, wystarczy dodać bardzo uproszczone przycinanie wierzchołków, ucinające ich generowanie, gdy Z jest wartością mniejszą lub bliską 0.

Translacja i obracanie

Translacja sceny o wektor [dx, dy, dz] polega na przemnożeniu wierzchołków przez macierz translacji T:

```
    [ 1 0 0 dx ]
T = [ 0 1 0 dy ]
    [ 0 0 1 dz ]
    [ 0 0 0  1 ]
```

Obracanie zostało zrealizowane analogicznie, za pomocą 3 macierzy rotacji 4x4 wokół wybranych osi układu współrzędnych o zadany kąt θ.

Dla obrotu wokół osi X:

$$Rx = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

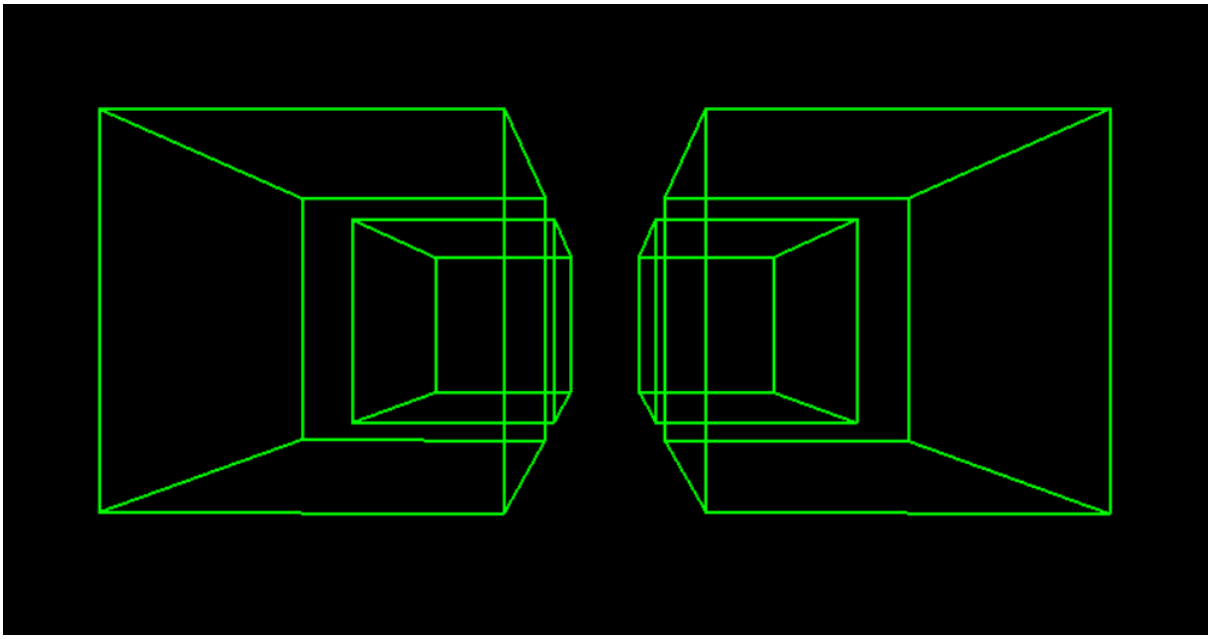
Dla obrotu wokół osi Y:

$$Ry = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

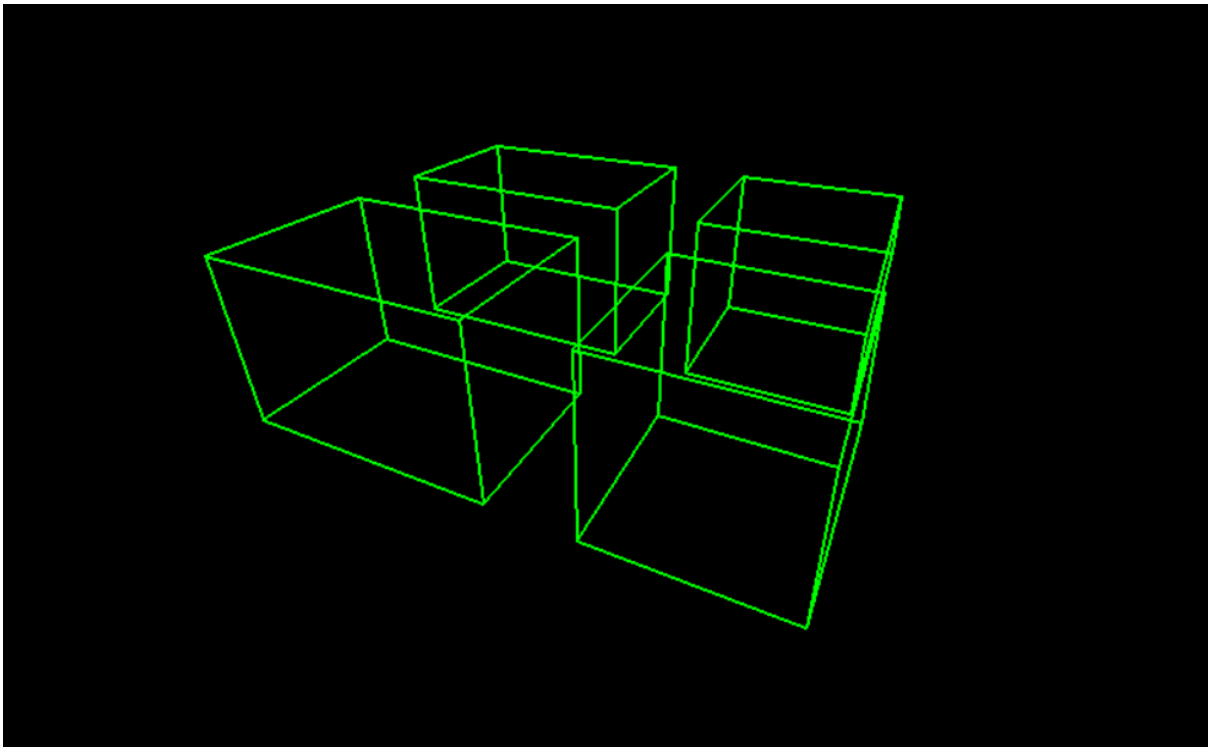
Dla obrotu wokół osi Z:

$$Rz = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

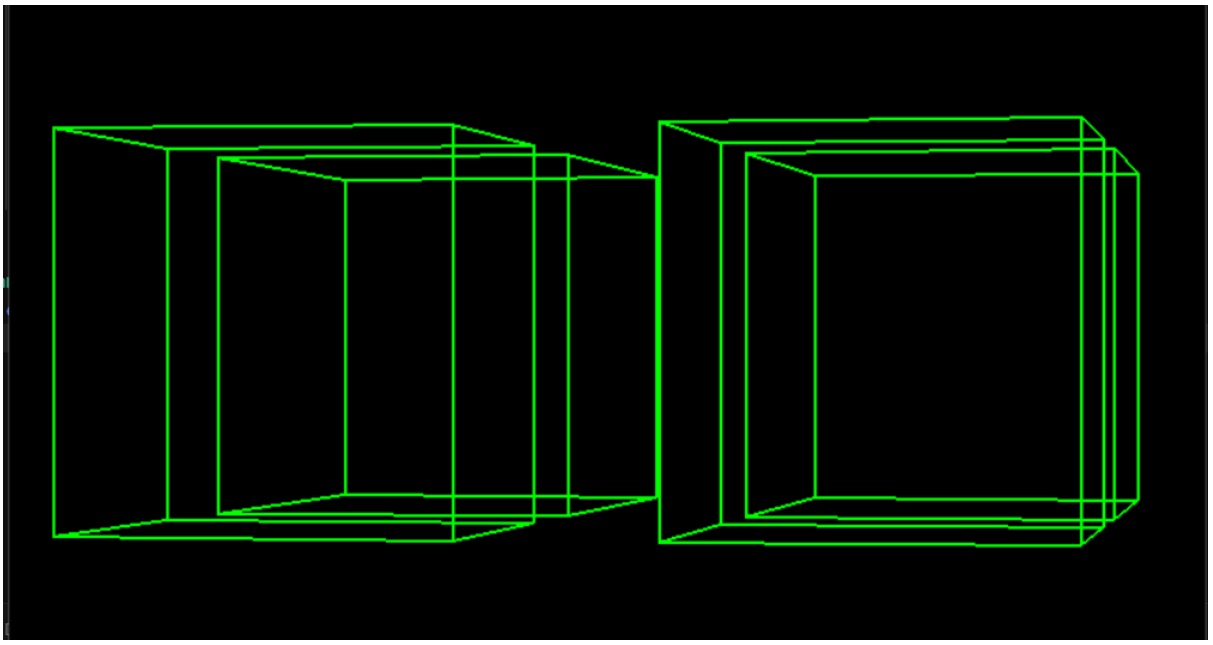
5. Testy



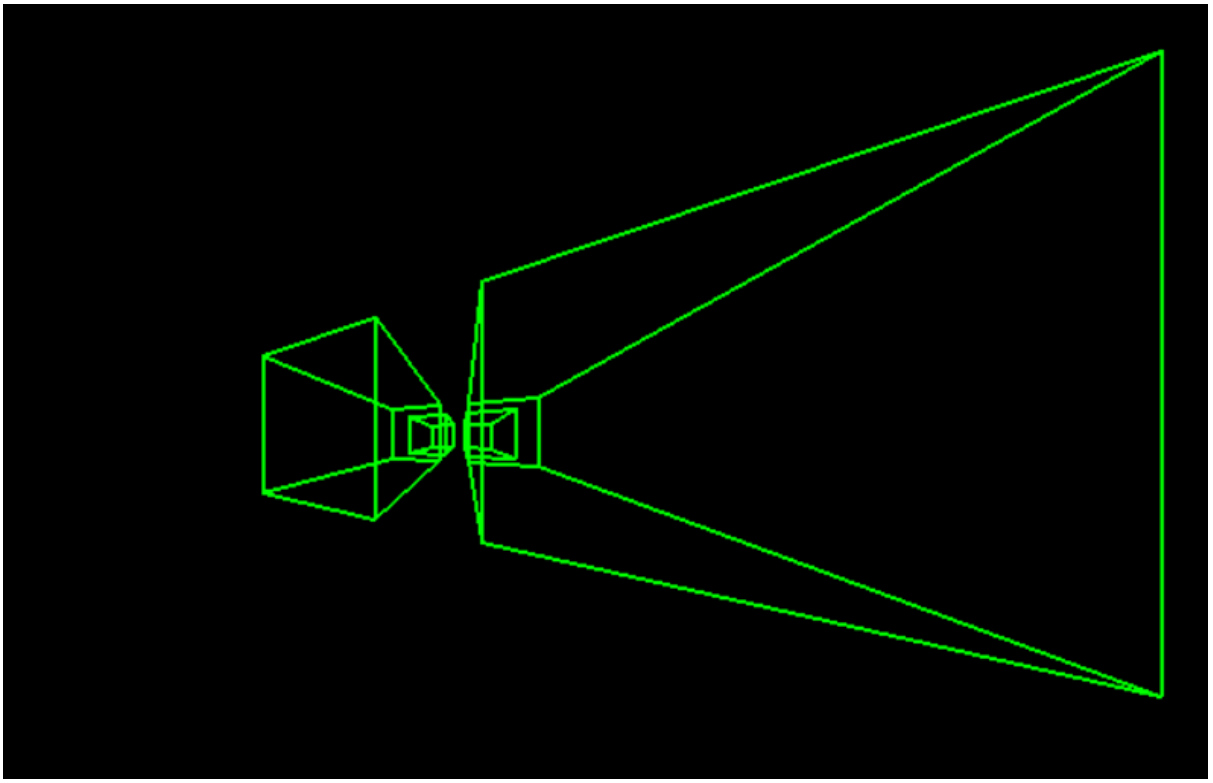
Rys. 1. Scena początkowa z lekką translacją w prawo.



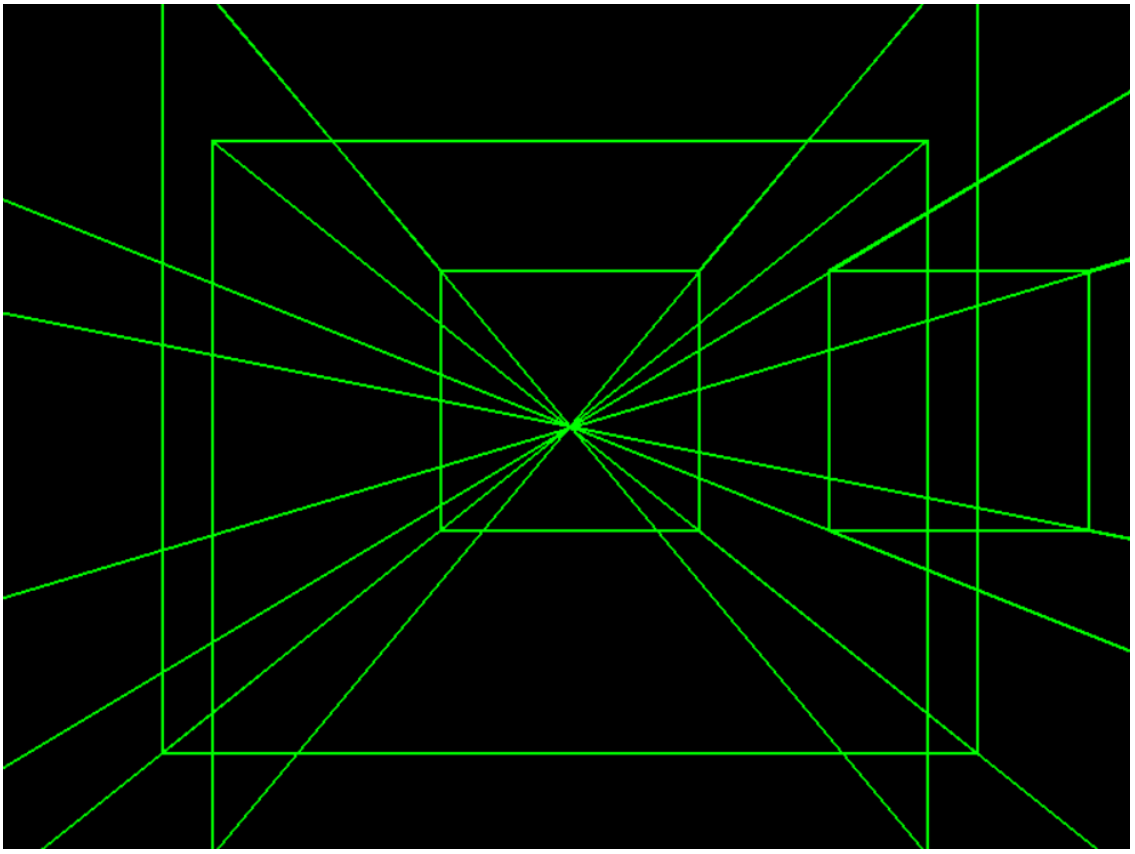
Rys. 2. Widok z lotu ptaka.



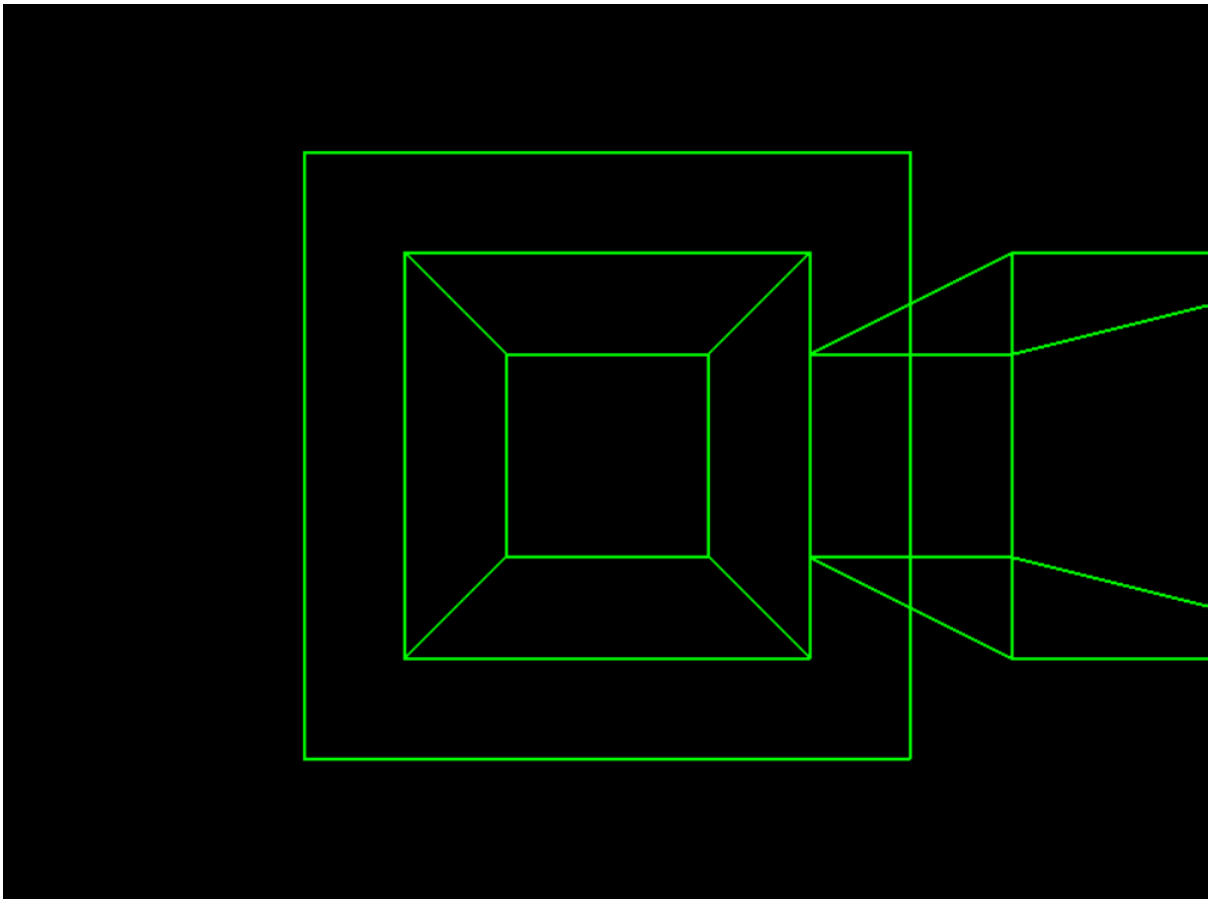
Rys. 3. Obiekty znajdują się daleko od kamery, duże przybliżenie



Rys. 4. Obiekty znajdują się blisko do kamery, duże oddalenie



Rys. 5. Scena wygenerowana dla wyłączonego przycinania, gdy Z wchodzi na ujemne wartości



Rys. 6. Ta sama scena wygenerowana dla włączonego uproszczonego przycinania, widać brak krawędzi dla wierzchołków, które znajdują się “za kamerą”.